

Consultas dinámicas: Specifications y paginación

7. Consultas dinámicas: Specifications y paginación

Consultas dinámicas: Specifications y paginación

Conoces el id

```
findById(42): el libro número 42
```

No conoces ningún id

«libros de menos de 20 € de una editorial»



Los filtros fijos no escalan

Cada usuario combina filtros distintos: un método por combinación crece sin control.

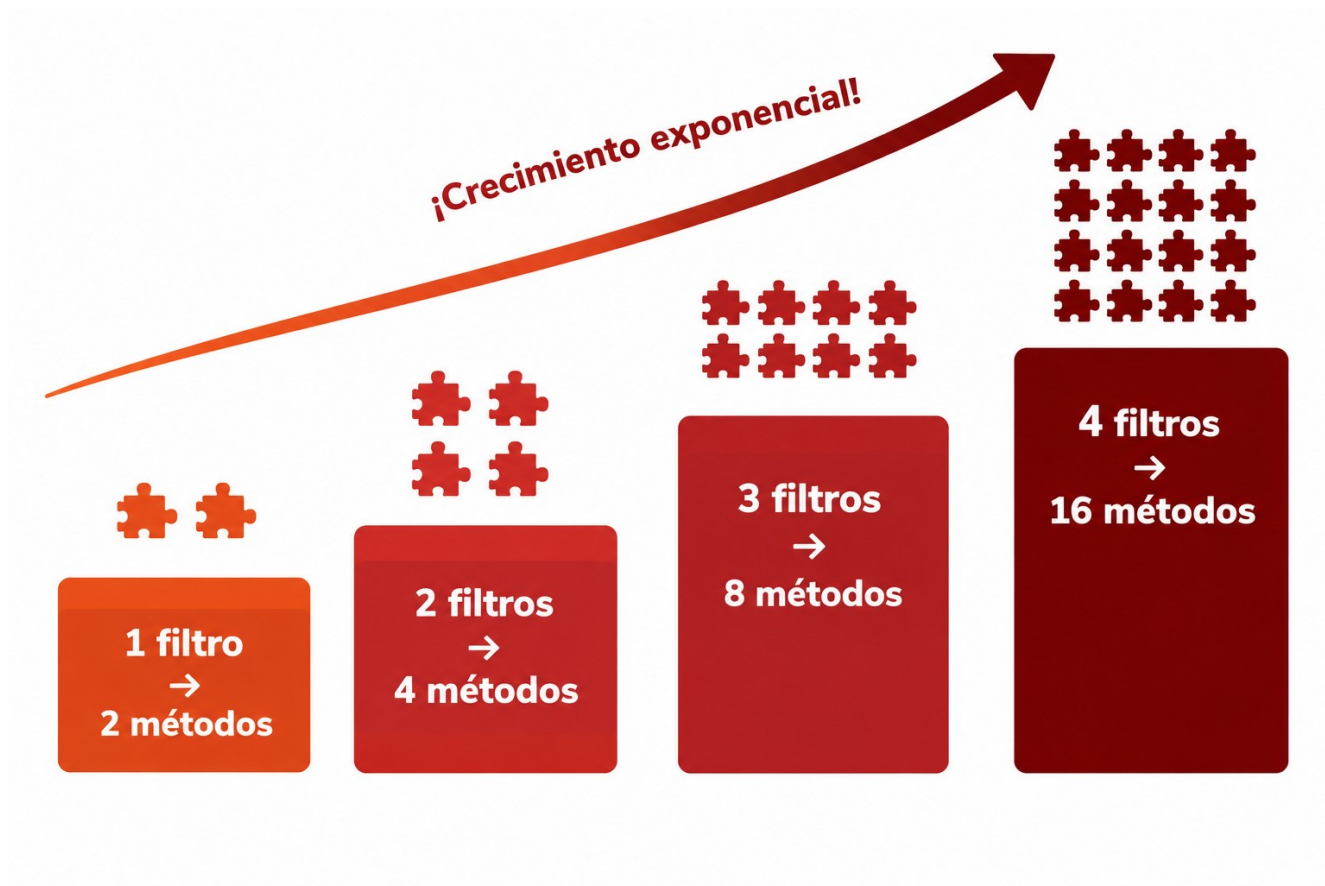


No puedes devolver todo de golpe

Ninguna API sería capaz de devolver un millón de filas en una sola respuesta.

Dos problemas distintos que comparten el mismo listado como escenario.

Problema A: los filtros fijos no escalan



1 filtro

```
findByTituloContaining(...)
```

2 filtros

```
...AndPrecioGreaterThanOrEqualTo(...)
```

3 filtros

```
...AndPrecioLessThanOrEqualTo(...)
```

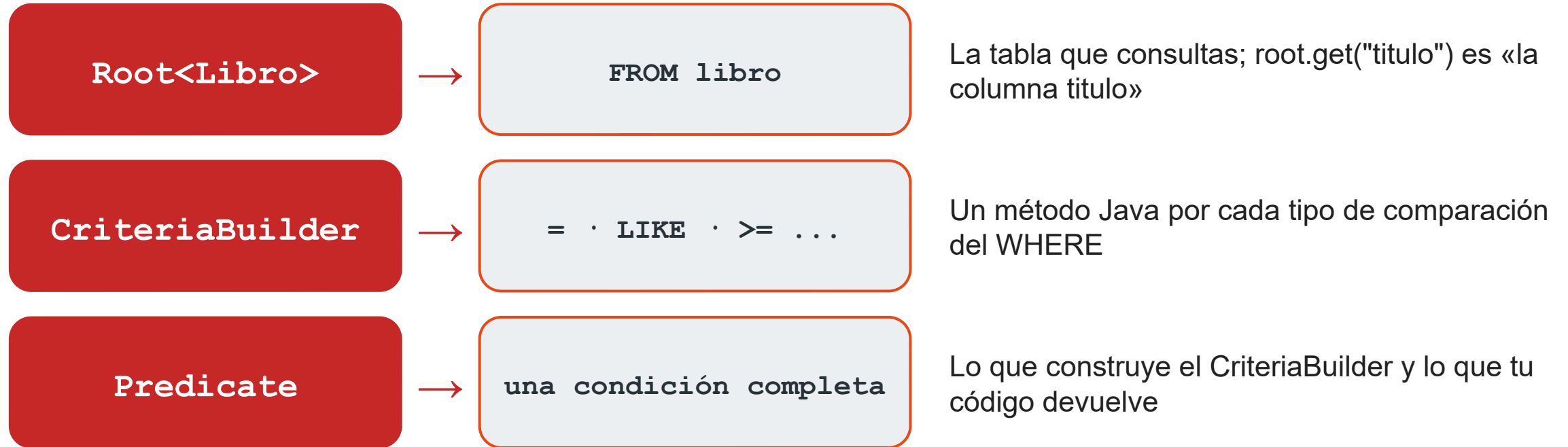
4 filtros

```
...AndEditorialId(...)
```

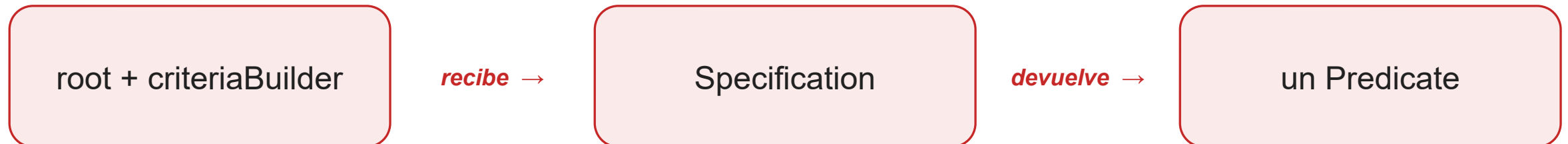
Cada filtro nuevo duplica los métodos, y el nombre deja de ser legible mucho antes de llegar a 16.

Método derivado por nombre: `findByPrecioLessThan(BigDecimal precio)` funciona mientras todos busquen siempre por lo mismo.

Las piezas nuevas, traducidas desde SQL



Una Specification<Libro> es, literalmente, una función:



Resolviendo el problema A: tu primera Specification

1. Habilitar Specifications en el repositorio

```
public interface LibroRepository extends
    JpaRepository<Libro, Long>,
    JpaSpecificationExecutor<Libro> {
}
```

2. La Specification, condición por condición

```
(root, query, criteriaBuilder) ->
    criteriaBuilder.like(
        criteriaBuilder.lower(root.get("titulo")),
        "%" + titulo.toLowerCase() + "%");
```

1

`root.get("titulo")`
es la columna titulo

2

`criteriaBuilder.lower(...)`
pasa la columna a minúsculas: la
búsqueda no distingue mayúsculas

3

`criteriaBuilder.like(...)`
construye el Predicate: LIKE
'%valor%', con métodos Java

El tercer parámetro, query, no se usa en filtros simples: la forma de la función lo exige.

El filtro es opcional: Specification.unrestricted()

```
public final class LibroSpecifications {  
  
    public static Specification<Libro>  
        tituloContiene(String titulo) {  
        if (titulo == null || titulo.isBlank()) {  
            return Specification.unrestricted();  
        }  
        return (root, query, criteriaBuilder) ->  
            criteriaBuilder.like(  
                criteriaBuilder.lower(root.get("titulo")),  
                "%" + titulo.toLowerCase() + "%");  
    }  
  
    // precioMayorOIgualA, precioMenorOIgualA,  
    // perteneceAEditorial... - mismo patrón.  
}
```

Filtro null

unrestricted(): ninguna condición añadida. Se salta de verdad.

Un valor real

El Predicate (LIKE, >=...): se añade al WHERE.

unrestricted() ya viene con Spring Data: al combinarla es como si no existiera.

4. El DTO de filtro y el service

```
public record LibroFiltroDTO(String titulo, BigDecimal precioMin,  
                             BigDecimal precioMax, Long editorialId) {}
```

```
@Transactional(readOnly = true)  
public List<LibroResponseDTO> findByFiltro(  
    LibroFiltroDTO filtro) {  
    Specification<Libro> spec = Specification  
        .where(LibroSpecifications  
            .tituloContiene(filtro.titulo()))  
        .and(LibroSpecifications  
            .precioMayorOIgualA(filtro.precioMin()));  
  
    return libroRepository.findAll(spec)  
        .stream()  
        .map(this::mapToDTO)  
        .toList();  
}
```

where (...) .and (...)

Encadena las piezas en una sola consulta. Las `unrestricted()` no añaden nada.

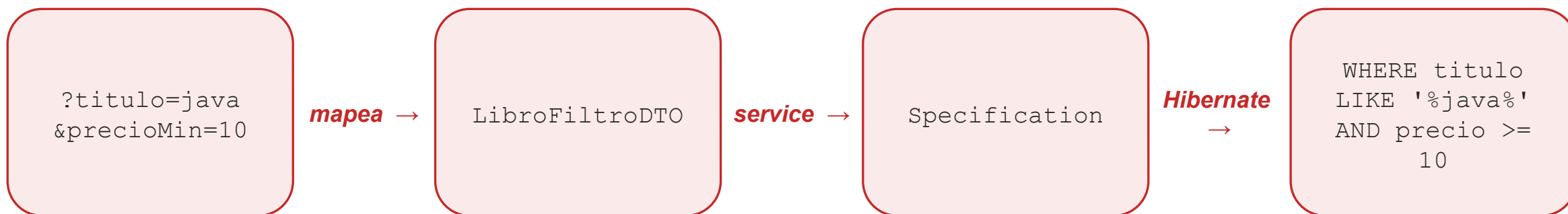
findAll(spec)

El `findAll` de siempre, con una `Specification`: devuelve todos los que cumplan, sin límite de página todavía.

5. El controller: de la URL al SQL

```
@GetMapping
public ResponseEntity<List<LibroResponseDTO>> getAll(@ModelAttribute LibroFiltroDTO filtro) {
    return ResponseEntity.ok(libroService.findByFiltro(filtro));
}
```

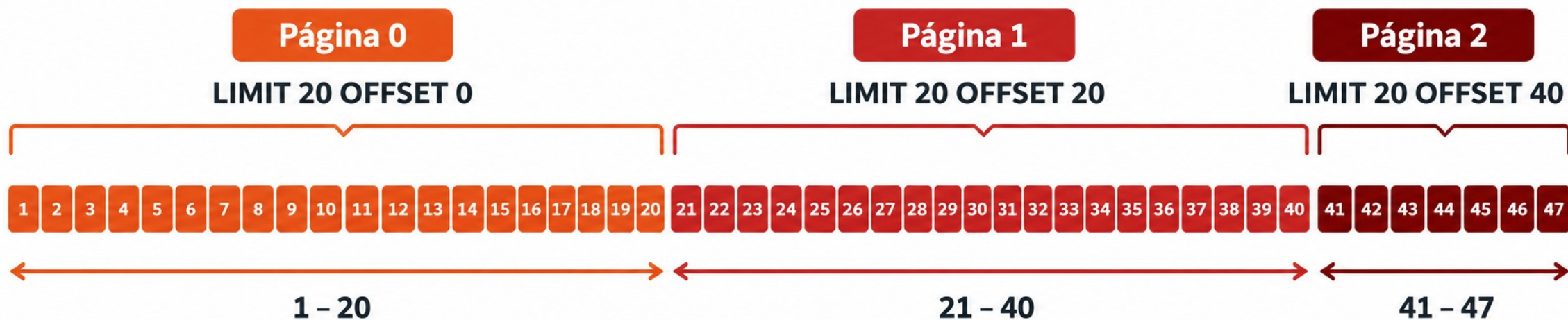
@ModelAttribute mapea la query string al DTO:



Problema A resuelto — pero con miles de filas, la respuesta seguiría trayéndolas todas de golpe. Ese es el problema B.

Problema B: tampoco puedes devolver todo de golpe

Como en Google: con miles de resultados, te da los primeros 10 y un botón para los siguientes. La paginación aplica esa idea a tu API.



La primera página es la 0, no la 1

Spring Data numera las páginas desde cero. Pedir la «página 1» te da la segunda.

Pageable y Page: la petición y la respuesta

Pageable: la petición

Qué página, de qué tamaño y con qué orden. Sin anotación: Spring lee page, size y sort de la query string.

Page<T>: la respuesta

El contenido de esa página, más metadatos: total de elementos, total de páginas...

Por debajo, Hibernate lo traduce a SQL de toda la vida:

```
GET .../libros?page=0
    &size=3
    &sort=titulo,asc
```

Spring →

```
Pageable
page 0 · size 3
```

Hibernate →

```
ORDER BY titulo
LIMIT 3 OFFSET 0
```

OFFSET = página × tamaño: la página 2 con tamaño 20 salta $2 \times 20 = 40$ filas, las de las páginas 0 y 1.

La respuesta de Page<T>, campo a campo

```
{
  "content": [ /* libros */ ],
  "totalElements": 7,
  "totalPages": 3,
  "number": 0,
  "size": 3,
  "numberOfElements": 3,
  "first": true,
  "last": false,
  "empty": false
  // pageable y sort: ver apuntes
web
}
```

content

Los elementos de esta página

totalElements

Filas totales de la consulta, no solo de esta página

totalPages

Páginas necesarias: $\text{totalElements} / \text{size}$, hacia arriba

number

La página actual (empieza en 0)

El resto es información derivada:

size

numberOfElements

first / last

Código completo: *apuntes web* — Tema 1, «Consultas dinámicas: Specifications y paginación».

La paginación sola: service y controller

Service: findAll(Pageable) ya viene en JpaRepository

```
@Transactional(readonly = true)
public Page<LibroResponseDTO>
    findAllPaginated(
        Pageable pageable) {
    return libroRepository
        .findAll(pageable)
        .map(this::mapToDTO);
}
```

Controller

```
@GetMapping
public ResponseEntity<Page<LibroResponseDTO>>
    getAll(@PageableDefault(size = 20)
        Pageable pageable) {
    return ResponseEntity.ok(
        libroService
            .findAllPaginated(pageable));
}
```

@PageableDefault(size = 20) fija el tamaño por defecto. Fíjate: getAll() ya no devuelve List, sino Page.

Un 500 que en realidad es culpa del cliente

Con sort=noExiste,asc responde 500, no 400. Se resolverá más adelante con GlobalExceptionHandler, en Programación de Servicios y Procesos.

Todo junto: Specifications + paginación

En tu proyecto real no habrá tres endpoints: habrá uno solo que hace ambas cosas a la vez.

Pieza	Solo Specifications	Solo paginación	Versión combinada
Repositorio	JpaSpecificationExecutor	Ninguno extra	JpaSpecificationExecutor
Service, devuelve	List<LibroResponseDTO>	Page<LibroResponseDTO>	Page<LibroResponseDTO>
Service, recibe	LibroFiltroDTO filtro	Pageable pageable	Los dos
findAll(...)	con spec	con pageable	con spec y pageable
Controller	@ModelAttribute filtro	@PageableDefault pageable	Los dos a la vez

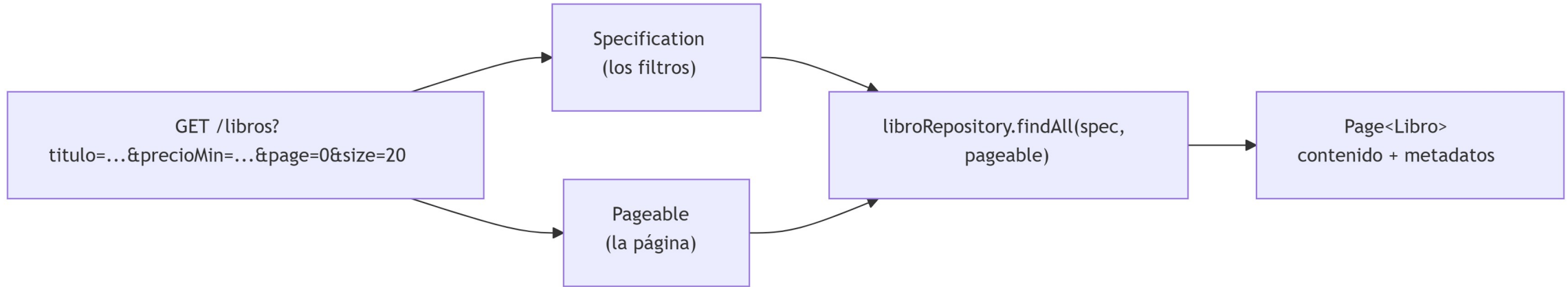
El service y el controller combinados

```
@Transactional(readOnly = true)
public Page<LibroResponseDTO> findAllPaginated(LibroFiltroDTO filtro, Pageable pageable) {
    Specification<Libro> spec = Specification
        .where(LibroSpecifications.tituloContiene(filtro.titulo()))
        .and(LibroSpecifications.precioMayorOIgualA(filtro.precioMin()))
        .and(LibroSpecifications.precioMenorOIgualA(filtro.precioMax()))
        .and(LibroSpecifications.perteneceAEditorial(filtro.editorialId()));
    return libroRepository.findAll(spec, pageable).map(this::mapToDTO);
}
```

```
@GetMapping
public ResponseEntity<Page<LibroResponseDTO>> getAll(
    @ModelAttribute LibroFiltroDTO filtro,
    @PageableDefault(size = 20) Pageable pageable) {
    return ResponseEntity.ok(libroService.findAllPaginated(filtro, pageable));
}
```

findAll(spec, pageable) aplica la Specification y trocea el resultado: una sola llamada resuelve los dos problemas.

El diagrama del principio y Swagger UI



@ParameterObject

Swagger UI puede mostrar un único cuadro con JSON en vez de un campo por filtro: springdoc no siempre «explota» @ModelAttribute ni Pageable. @ParameterObject lo arregla.

```
@ParameterObject @ModelAttribute
    LibroFiltroDTO filtro,
@ParameterObject
@PageableDefault(size = 20)
    Pageable pageable
```

Ideas clave

1

`findById` + `save()` sirven para un objeto con id conocido; no para buscar por criterios variables.

2

Problema A (filtros que no escalan) y problema B (no devolver todo de golpe) son distintos y comparten listado.

3

Una `Specification` construye la consulta con piezas combinables; `unrestricted()` es la pieza vacía para un filtro null.

4

Paginación: `Pageable` es la petición y `Page<T>` la respuesta. Páginas desde 0; por debajo, `LIMIT/OFFSET`.

5

`JpaSpecificationExecutor` habilita `Specifications`; `findAll(spec)`, `findAll(pageable)` y `findAll(spec, pageable)` son sobrecargas.

6

`@ParameterObject` hace que Swagger UI muestre un campo por atributo, no un cuadro con JSON.